GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

LABORATORY REPORT

AGILE SOFTWARE DEVELOPMENT LAB

Lab No. 2

BESE III YEAR, VI-SEM

Submitted By:

Prabin Thapa

Submitted To:

Er. Rajendra Bahadur Thapa

Date: July 4, 2026

NAME OF EXPERIMENT: Implementing different features of IDE (VSCODE or any)

AIM: To understand and implement the various features of an Integrated Development Environment (IDE), such as Visual Studio Code (VS Code), for efficient software development, including code editing, debugging, extensions, version control integration, terminal usage, and project management.

Objectives:

1. To understand the concept and importance of an Integrated Development Environment (IDE).
2. To explore the user interface and components of Visual Studio Code (or any IDE).
3. To create and manage software development projects using the IDE.
4. To use features such as syntax highlighting, IntelliSense, debugging, integrated terminal, and extensions.
5. To integrate Git and GitHub within the IDE.
6. To improve coding efficiency through productivity features like code formatting, snippets, and auto-completion.
7. To understand how an IDE simplifies software development and debugging..

THEORY:

Introduction to IDE

An Integrated Development Environment (IDE) is a software application that provides all the essential tools required for software development in a single interface. It combines a source code editor, compiler/interpreter, debugger, terminal, and other development tools, making programming faster and more efficient.

Popular IDEs include Visual Studio Code (VS Code), Visual Studio, Eclipse, IntelliJ IDEA, NetBeans, and PyCharm. Among these, Visual Studio Code is widely used because it is lightweight, free, highly customizable, and supports multiple programming languages through extensions.

Features of Visual Studio Code

Features of an IDE

1. Build (Debug and Release)

An IDE allows developers to build applications in different modes:

- **Debug Build:** Compiles the program with debugging information, making it easier to locate and fix errors during development.
- **Release Build:** Creates an optimized version of the application for deployment by removing debugging information and improving performance.

2. Breakpoints

A **breakpoint** is a debugging feature that temporarily pauses program execution at a specified line of code. This enables developers to inspect variables, monitor program flow, and identify logical or runtime errors before continuing execution.

3. Refactoring

Refactoring is the process of improving the internal structure of source code without changing its functionality. IDEs provide automated refactoring tools such as renaming variables, methods, or classes, extracting methods, and reorganizing code to improve readability and maintainability.

4. Source Code Collaboration (Git Integration)

Modern IDEs provide built-in support for **Git**, allowing developers to manage source code versions and collaborate effectively. Common operations include repository initialization, committing changes, branching, merging, pulling updates, and pushing code to remote repositories such as GitHub.

5. Palette Code Generation (Command Palette)

The **Command Palette** in Visual Studio Code provides quick access to almost every IDE function through a searchable interface. Developers can create files, run commands, change settings, install extensions, execute builds, and perform many other tasks without navigating through menus, improving productivity.

6. IntelliSense

IntelliSense is an intelligent code-completion feature that provides suggestions for variables, functions, classes, methods, parameters, and keywords while coding. It also displays documentation and detects syntax errors, helping developers write code faster and with fewer mistakes.

Advantages of Using an IDE

- Increases programming productivity.
- Simplifies debugging and testing.
- Reduces syntax errors through code suggestions.
- Provides integrated version control support.
- Makes project management easier.
- Supports multiple programming languages and extensions.

Algorithm

1. Install Visual Studio Code (or any IDE).
2. Launch the IDE.
3. Create or open a project folder.
4. Create a source code file.
5. Write a simple program.
6. Observe syntax highlighting and IntelliSense suggestions.
7. Open the integrated terminal and execute the program.
8. Install the required language extensions.
9. Format the source code using the formatter.
10. Set breakpoints in the program.
11. Run the debugger and inspect variables.
12. Use the Search feature to locate text in the project.
13. Initialize a Git repository and commit the project using the Source Control panel.
14. Save all files.
15. Close the project.
16. End.

CODE:

Program (Python)

```
def calculate_sum(a, b):
    return a + b
def main():
    num1 = 10
    num2 = 20
    result = calculate_sum(num1, num2)
    print(f"Number 1 = {num1}")
    print(f"Number 2 = {num2}")
    print(f"Sum = {result}")


if __name__ == "__main__":
    main()
```

Output:

```
Number 1 = 10
Number 2 = 20
Sum = 30
```

Demonstration of IDE Features

1. Build (Debug and Run)

Although Python is an interpreted language and does not have a traditional Release Build like C/C++, VS Code supports Run and Debug execution.

Debug Mode

- Open the Python file in VS Code.
- Click Run and Debug or press F5.
- The program runs with debugging support.

Output:

```
Starting Debugger...
Number 1 = 10
Number 2 = 20
Sum = 30

Program finished successfully.
```

Normal Run

Run the program from the integrated terminal**:**

```
python lab2.py
```

Output

2. Breakpoint

```
Number 1 = 10
result = calculate_sum(num1, num2)
Sum = 30
```

Place a breakpoint on the following line:

Run the debugger (F5).

Execution pauses before calculating the result.

Debugger Output:

```
Breakpoint Hit
num1 = 10
num2 = 20
result = Not Assigned
Continue Execution...
```

After continuing:

```
Number 1 = 10
Number 2 = 20
Sum = 30
```

3. Refactoring

VS Code provides Rename Symbol (F2).

Original function:

```
def calculate_sum(a, b):
```

Rename it to:

The IDE automatically updates all references.

```
def add_numbers(a, b):
result = add_numbers(num1, num2)
```

Updated code:

Output

```
Number 1 = 10
Number 2 = 20
Sum = 30
```

4. Source Code Collaboration (Git)

Open the integrated terminal and execute:

```
git init
git add .
git commit -m "Initial Commit"
```

Push to GitHub:

```
git remote add origin https://github.com/prabin-maqar/lab2.git
git push -u origin main
```

Output

```
Initialized empty Git repository
[main (root-commit)] Initial Commit
Successfully pushed to GitHub.
```

5. Command Palette

Open the Command Palette using:

Ctrl + Shift + P

Example commands:

> Python: Select Interpreter
> Format Document
> Git: Clone
> Run Python File
> Reload Window

Output

Command executed successfully.

6. IntelliSense

While typing:

```
pri
```

VS Code automatically suggests:

```
print()
```

Typing:

```
calculate_
```

shows:

```
calculate_sum(a, b)
```

Hovering over the function displays:

```
calculate_sum(a: int, b: int)

Returns the sum of two numbers.
```

Output

IntelliSense suggestions displayed successfully.

Create the file:

RESULT:

The various features of Visual Studio Code, including Run/Debug execution, Breakpoints, Refactoring, Git-based Source Code Collaboration, Command Palette, and IntelliSense, were successfully implemented and demonstrated using a Python program.

Since Python is an interpreted language, it does not have separate Debug Build and Release Build modes like compiled languages such as C, C++, or Java. The IDE's Run and Debug modes were used to demonstrate program execution and debugging.

CONCLUSION:

This experiment provided practical experience with the essential features of an Integrated Development Environment using Visual Studio Code. The IDE streamlined software development by integrating code editing, debugging, terminal access, version control, and extension support into a single platform. Utilizing these features improved coding efficiency, reduced development time, simplified debugging, and enhanced project management. Overall, Visual Studio Code proved to be a powerful and user-friendly development environment suitable for modern software engineering across multiple programming languages and platforms.